RMS code:

```
#include <stdio.h>

#define MAX 10

typedef struct {
    int pid;
    int burst;
    int period;
} Task;

// GCD
int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

// LCM
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

// Find hyperperiod
int find_lcm(Task t[], int n) {
    int res = t[0].period;
    for (int i = 1; i < n; i++) {
        res = lcm(res, t[i].period);
    }
    return res;
}

// Sort by period (RMS priority)
void sort(Task t[], int n) {
    Task temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (t[i].period > t[j].period) {
```

```
                temp = t[i];
                t[i] = t[j];
                t[j] = temp;
            }
        }
    }
}

int main() {
    printf("riya gupta,USN 1WA24CS235\n");
    int n;
    Task t[MAX];

    printf("Enter the number of processes:");
    scanf("%d", &n);

    printf("Enter the CPU burst times:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &t[i].burst);
        t[i].pid = i + 1;
    }

    printf("Enter the time periods:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &t[i].period);
    }

    // Sort tasks
    sort(t, n);

    int hyper = find_lcm(t, n);
    printf("LCM=%d\n\n", hyper);

    // Print table
    printf("Rate Monotone Scheduling:\n");
    printf("PID\tBurst\tPeriod\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\n", t[i].pid, t[i].burst, t[i].period);
    }

    // Utilization
    float util = 0;
    for (int i = 0; i < n; i++) {
        util += (float)t[i].burst / t[i].period;
```

```
}

// Predefined Liu & Layland bounds (no math.h needed)
float bound;
if (n == 1) bound = 1.0;
else if (n == 2) bound = 0.828427;
else if (n == 3) bound = 0.779763;
else if (n == 4) bound = 0.756828;
else bound = 0.693; // approximation

printf("\n%f <= %f => %s\n", util, bound, (util <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n\n", hyper);

// RMS Scheduling
int remaining[MAX] = {0};
int prev = -2;

for (int time = 0; time < hyper; time++) {

    // Release tasks
    for (int i = 0; i < n; i++) {
        if (time % t[i].period == 0) {
            remaining[i] = t[i].burst;
        }
    }

    int running = -1;

    // Select highest priority
    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            running = i;
            break;
        }
    }

    // Print only when task changes
    if (running != prev) {
        if (running == -1)
            printf("%dms onwards: CPU is idle\n", time);
        else
            printf("%dms onwards: Process %d running\n", time, t[running].pid);
```

```
            prev = running;
        }

        // Execute
        if (running != -1) {
            remaining[running]--;
        }
    }

    return 0;
}
```

Output

```
riya gupta,USN 1WA24CS235
Enter the number of processes:2
Enter the CPU burst times:
20
35
Enter the time periods:
50
100
LCM=100

Rate Monotone Scheduling:
PID Burst   Period
1   20  50
2   35  100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle


=== Code Execution Successful ===
```

EDF CODE:

```
#include <stdio.h>
#define MAX 10
int main() {
    printf("riya gupta,usn:1WA24CS235\n");
    int n;
    int burst[MAX], deadline[MAX], period[MAX];
    int remaining[MAX];
    int time = 0;
    printf("Enter number of tasks: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Task %d (burst deadline period): ", i + 1);
        scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);
        remaining[i] = 0;  // initially no job released
    }
    printf("\nPID\tBurst\tDeadline\tPeriod\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t\t%d\n", i + 1, burst[i], deadline[i], period[i]);
    }
    int limit = 20;
    printf("\nScheduling occurs for %d ms\n\n", limit);

    while (time < limit) {
        for (int i = 0; i < n; i++) {
            if (time % period[i] == 0) {
                remaining[i] = burst[i];
            }
        }
        int min_deadline = 9999;
        int selected = -1;
        for (int i = 0; i < n; i++) {
            if (remaining[i] > 0) {
                if (deadline[i] < min_deadline) {
                    min_deadline = deadline[i];
```

```
                selected = i;
            }
        }
    }
    if (selected == -1) {
        printf("%dms : CPU is idle.\n", time);
    } else {
        printf("%dms : Task %d is running.\n", time, selected + 1);
        remaining[selected]--;
    }
    time++;
  }
  return 0;
}
```

Output

```
riya gupta,usn:1WA24CS235
Enter number of tasks: 3
Task 1 (burst deadline period): 2 4 5
Task 2 (burst deadline period): 3 7 20
Task 3 (burst deadline period): 2 8 10

PID Burst   Deadline    Period
1   2   4       5
2   3   7       20
3   2   8       10

Scheduling occurs for 20 ms

0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 1 is running.
6ms : Task 1 is running.s
7ms : Task 3 is running.
8ms : Task 3 is running.
9ms : CPU is idle.
10ms : Task 1 is running.
11ms : Task 1 is running.
12ms : Task 3 is running.
13ms : Task 3 is running.
14ms : CPU is idle.
15ms : Task 1 is running.
16ms : Task 1 is running.
17ms : CPU is idle.
18ms : CPU is idle.
19ms : CPU is idle.


=== Code Execution Successful ===
```